Порождающие паттерны. Абстрактная фабрика

Порождающие паттерны

Порождающие паттерны проектирования описывают процесс создания объектов-продуктов определенных классов, при этом делая абстрактным сам процесс создания (*абстрагируют процесс инстанцирования*).

Абстрагирование процесса инстанцирования – это абстрактное описание процесса порождения экземпляра типа, через использование абстрактных фабричных методов.

```
class Program
{
    static void Main()
    {
        AbstractFactory factory = null;
        // Абстрагирование процесса инстанцирования.
        AbstractProduct product = factory.CreateProduct();
        // Абстрагирование вариантов использования.
        product.Operation();
    }
}
```

```
abstract class AbstractProduct
{
    public abstract void Operation();
}
```

```
abstract class AbstractFactory
{
    public abstract AbstractProduct CreateProduct();
}
```

Шаги реализации порождающих паттернов

1. Создаются абстрактные классы.
2. С их помощью задаются нужные типы, обобщающие собирательные характеристики.
3. Описываются интерфейсы взаимодействия с каждым типом.
4. Абстрактно (без реализации) описываются процессы порождения экземпляров этих типов и варианты использования этих типов, через имеющиеся у них интерфейсы.
5. Выбираются структуры данных и алгоритмы их обработки.
6. Выбираются детали реализации конкретных классов

Порождающие паттерны

```
class Program
{
    static void Main()
    {
        AbstractFactory factory = new ConcreteFactory();
        // Абстрагирование процесса инстанцирования.
        AbstractProduct product = factory.CreateProduct();
        // Абстрагирование вариантов использования.
        product.Operation();
    }
}
```

```
abstract class AbstractProduct
{
    public abstract void Operation();
}
```

```
abstract class AbstractFactory
{
    public abstract AbstractProduct CreateProduct();
}
```

```
class ConcreteProduct : AbstractProduct
{
    public override void Operation()
    {
        Console.WriteLine("Some work ...");
    }
}
```

```
class ConcreteFactory : AbstractFactory
{
    public override AbstractProduct CreateProduct()
    {
        return new ConcreteProduct();
    }
}
```

Порождающие паттерны

Порождающие паттерны проектирования используя технику абстрагирования процесса инстанцирования, помогают построить программную систему так, что эта система не зависит от способа создания в ней объектов, *композиции* (составления и соединения) объектов и *представления* (внутреннего состояния) объектов.

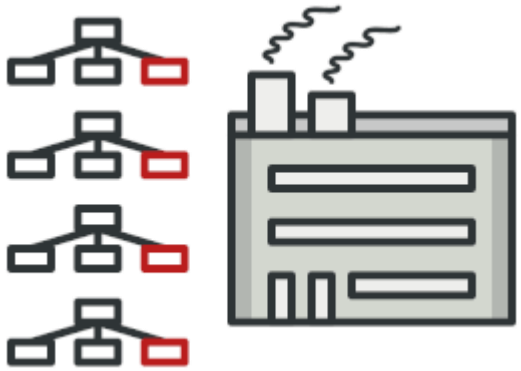
- *Порождающий паттерн уровня классов*, использует наследование, чтобы варьировать (видоизменять) инстанцируемый класс,
- *Порождающий паттерн уровня объектов*, использует композицию делегируя (передавая ответственность) инстанцирование другому объекту.

Использование порождающих паттернов оказываются полезным, когда в программной системе *чаще используется композиция объектов чем наследование классов*.

- *При использовании наследования* основной акцент делается на жестком кодировании фиксированного набора поведений (методов).
- В случае использования композиции можно получать любое число новых поведений (методов) составленных из уже существующих поведений.

Порождающие паттерны позволяют скрыть работу с конкретными классами и детали того как эти классы создаются и стыкуются. Единственная известная информация об объектах - это интерфейсы этих объектов, заданные через абстрактные классы. Таким образом упрощается понимание процессов создания объектов: когда, как, кто и что создает.

Паттерн Абстрактная Фабрика
Abstract Factory



Паттерн Abstract Factory - предоставляет клиенту интерфейс (набор методов) для создания семейств взаимосвязанных или взаимозависимых объектов-продуктов, при этом скрывает от клиента информацию о конкретных классах создаваемых объектов-продуктов.

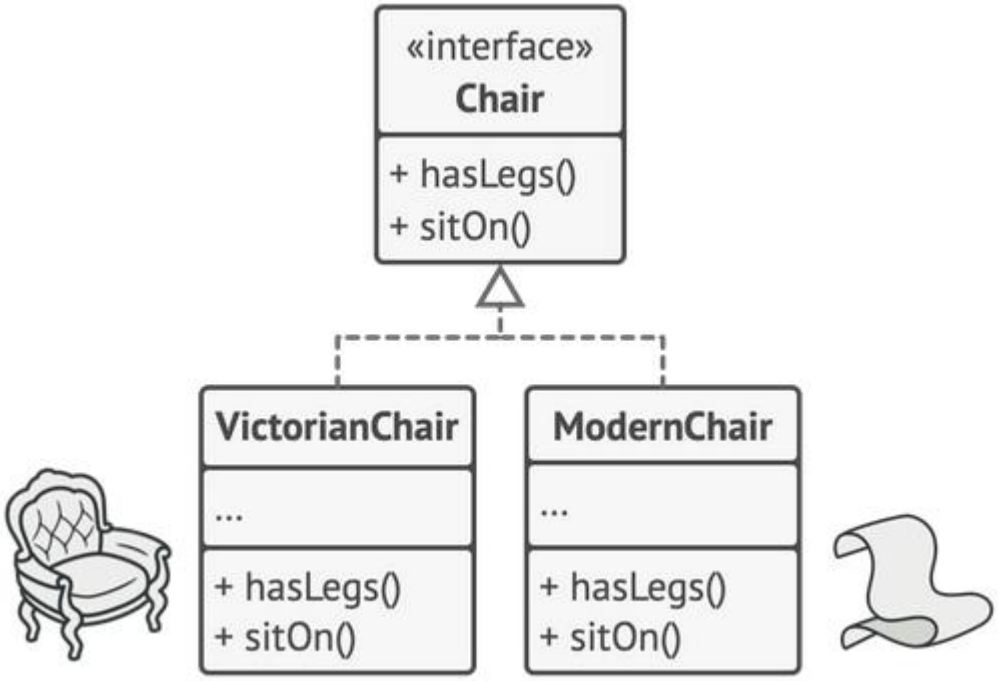
Абстрактная Фабрика. Пример 1

Мебельный магазин



Семейства продуктов и их вариации.

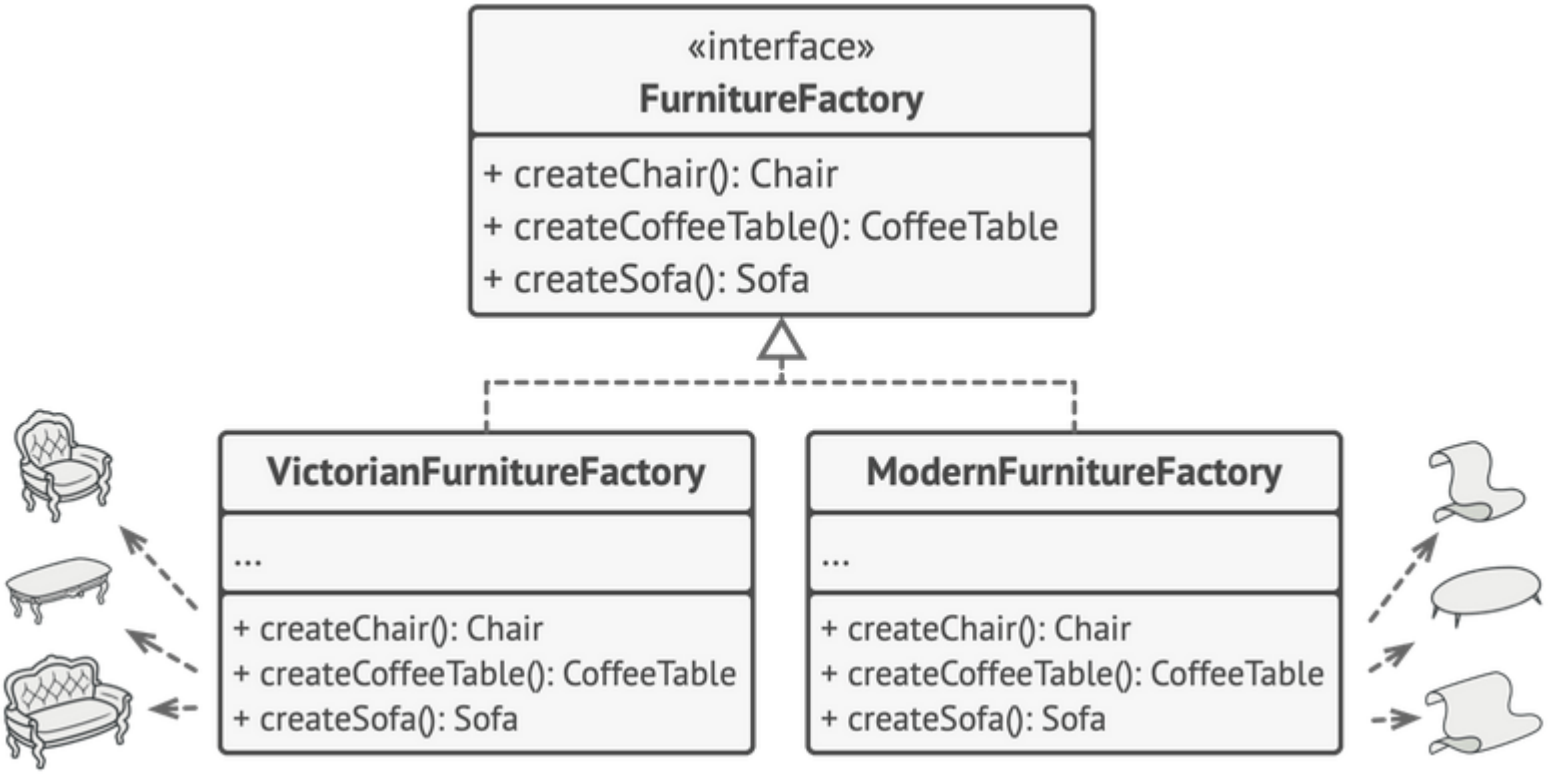
Абстрактная фабрика позволяет выделить общие интерфейсы для отдельных продуктов, составляющих семейства.
Т.е. все вариации (варианты) кресел получат общий интерфейс ***Кресло***, все диваны реализуют интерфейс **Диван** и т.д.



Все вариации одного и того же объекта должны быть в одной иерархии классов.

Создается *Абстрактная фабрика* — общий интерфейс, который содержит методы создания всех продуктов семейства (*создатьКресло*, *создатьДиван* и *создатьСтолик…*).

Эти операции должны возвращать абстрактные типы продуктов, представленные созданными интерфейсами — ***Кресла***, ***Диваны*** *и* ***Столики***

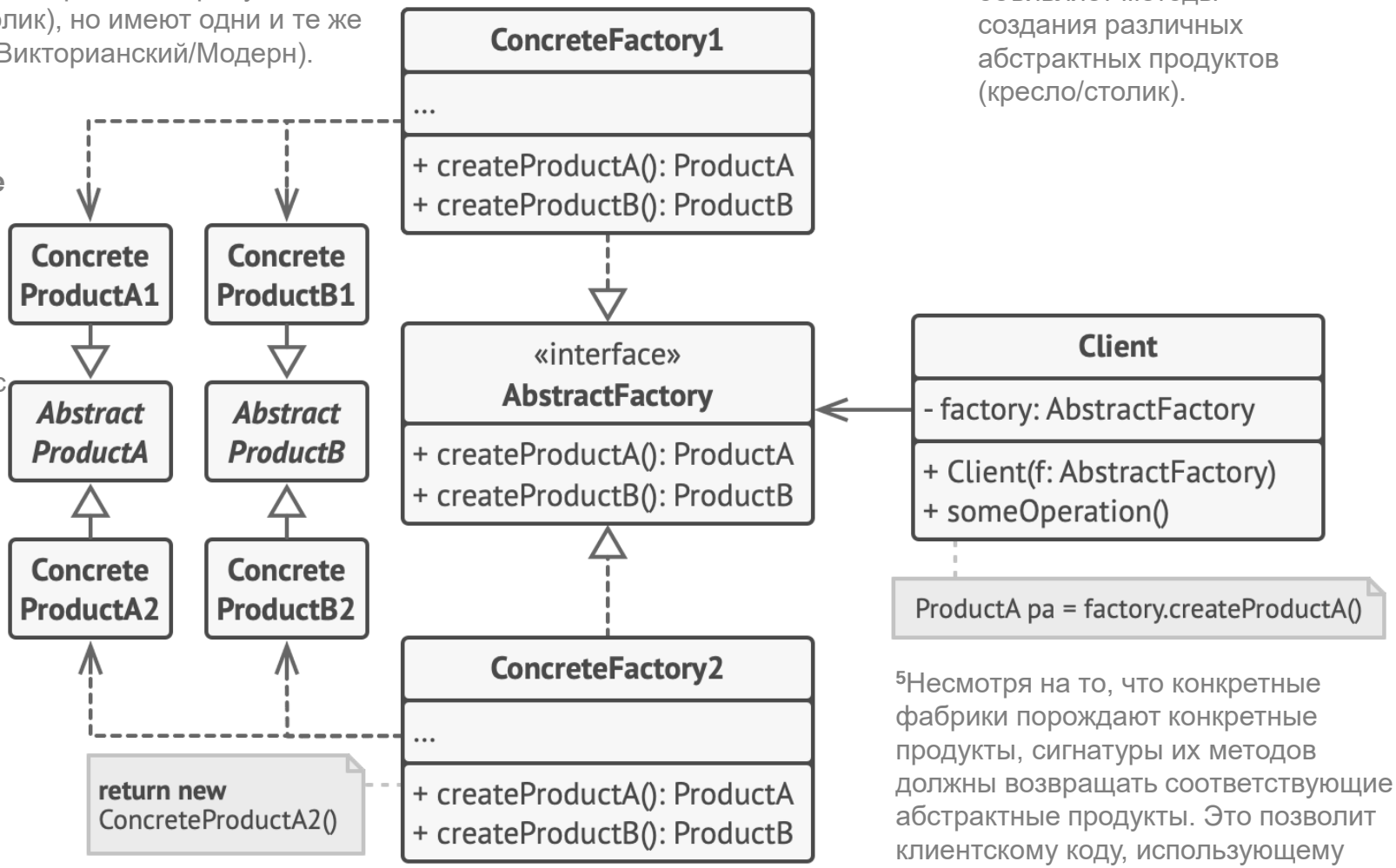


Конкретные фабрики соответствуют определённой вариации семейства продуктов.

[2]**Конкретные продукты** — большой набор классов, которые относятся к различным абстрактным продуктам (кресло/столик), но имеют одни и те же вариации (Викторианский/Модерн).

[3]**Абстрактная фабрика** объявляет методы создания различных абстрактных продуктов (кресло/столик).

[1]**Абстрактные продукты** объявляют интерфейсы продуктов, которые связаны друг с другом по смыслу, но выполняют разные функции.

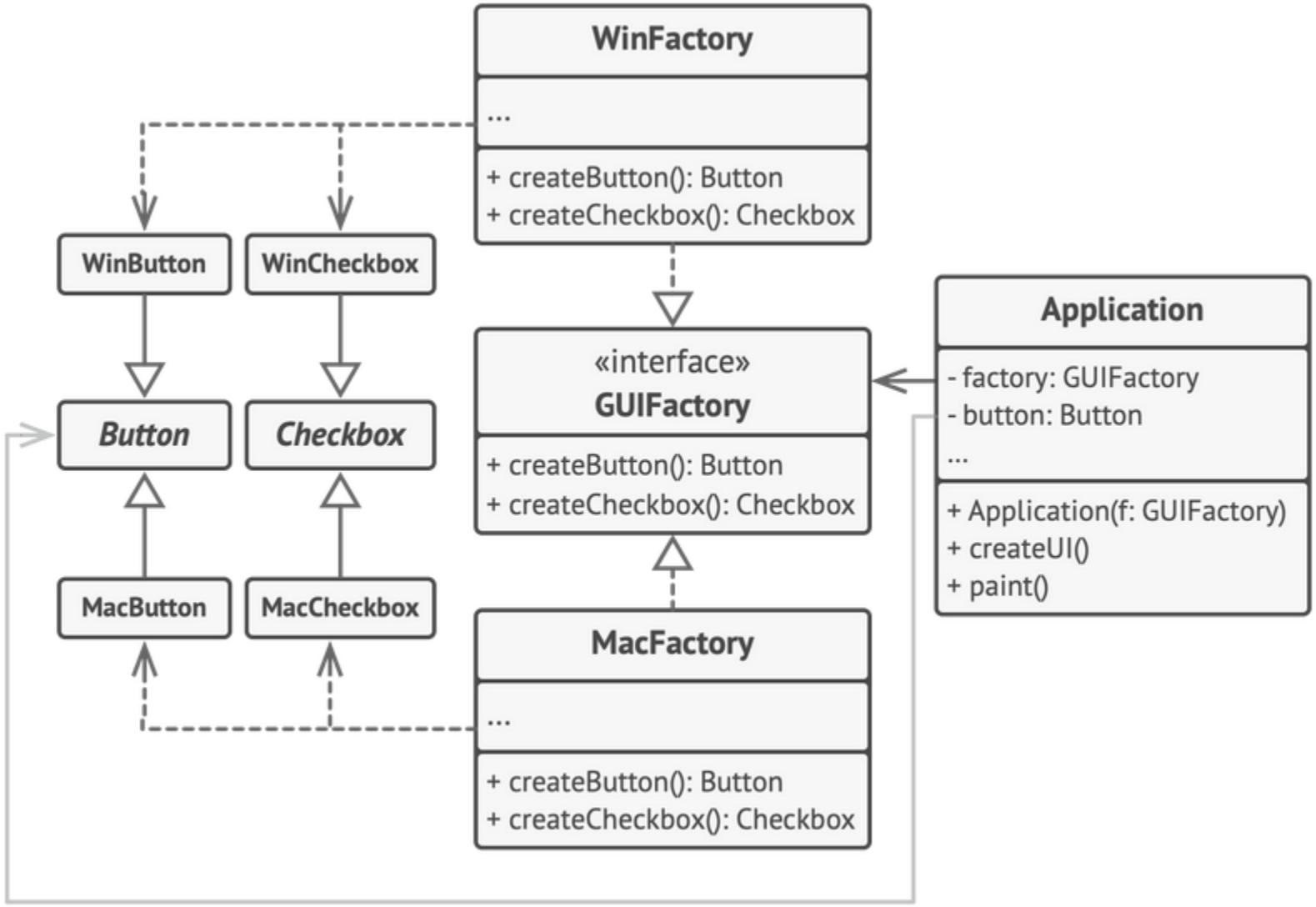


[4]**Конкретные фабрики** относятся каждая к своей вариации продуктов (Викторианский/Модерн) и реализуют методы абстрактной фабрики, позволяя создавать все продукты определённой вариации.

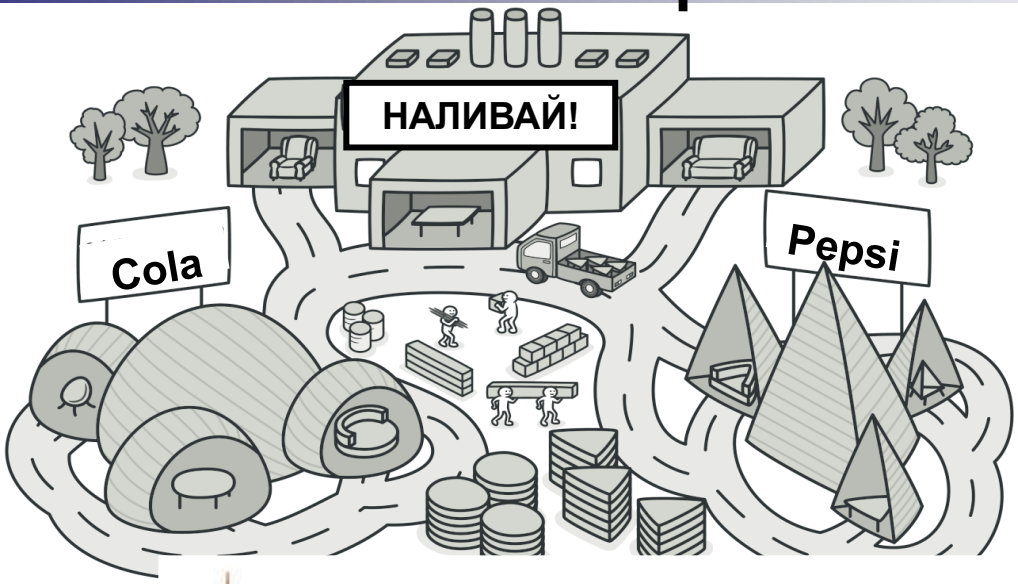
[5]Несмотря на то, что конкретные фабрики порождают конкретные продукты, сигнатуры их методов должны возвращать соответствующие абстрактные продукты. Это позволит клиентскому коду, использующему фабрику, не привязываться к конкретным классам продуктов. Клиент сможет работать с любыми вариациями продуктов через абстрактные интерфейсы.

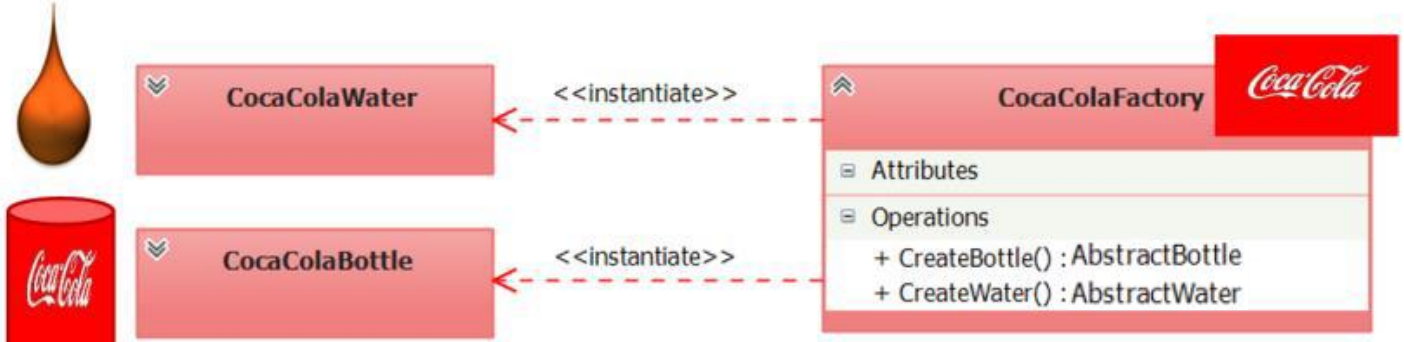
Абстрактная Фабрика. Пример 2

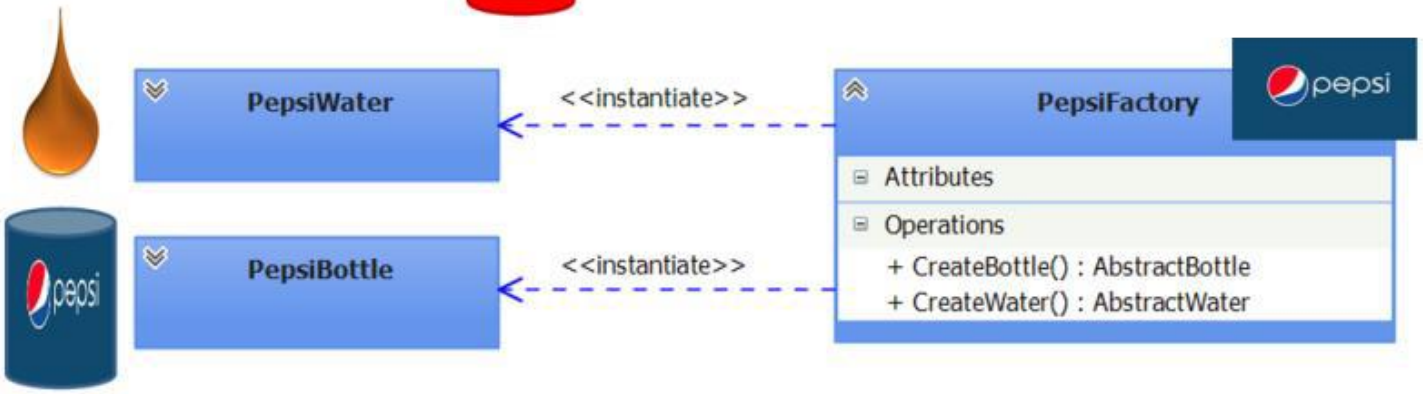
Абстрактная фабрика используется для создания кросс-платформенных элементов интерфейса



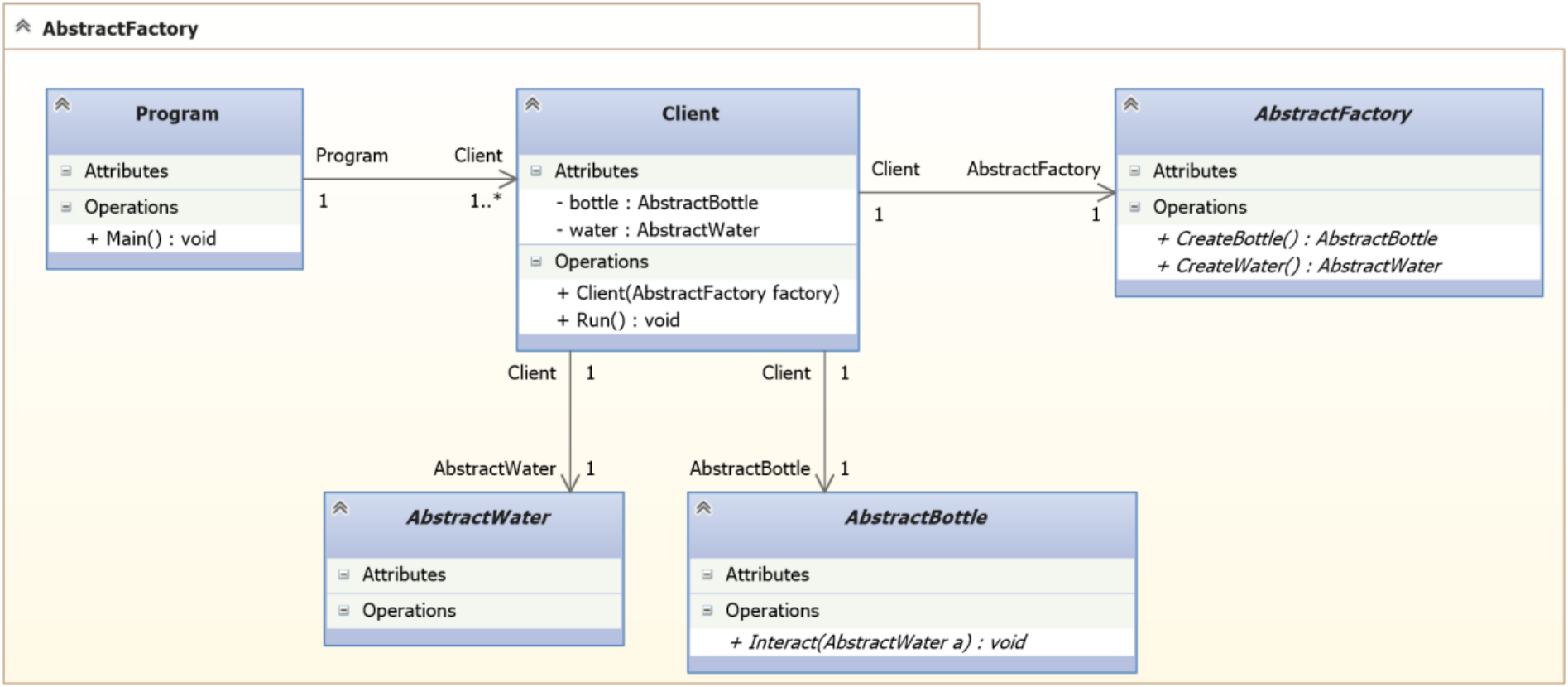
Абстрактная Фабрика. Пример 3







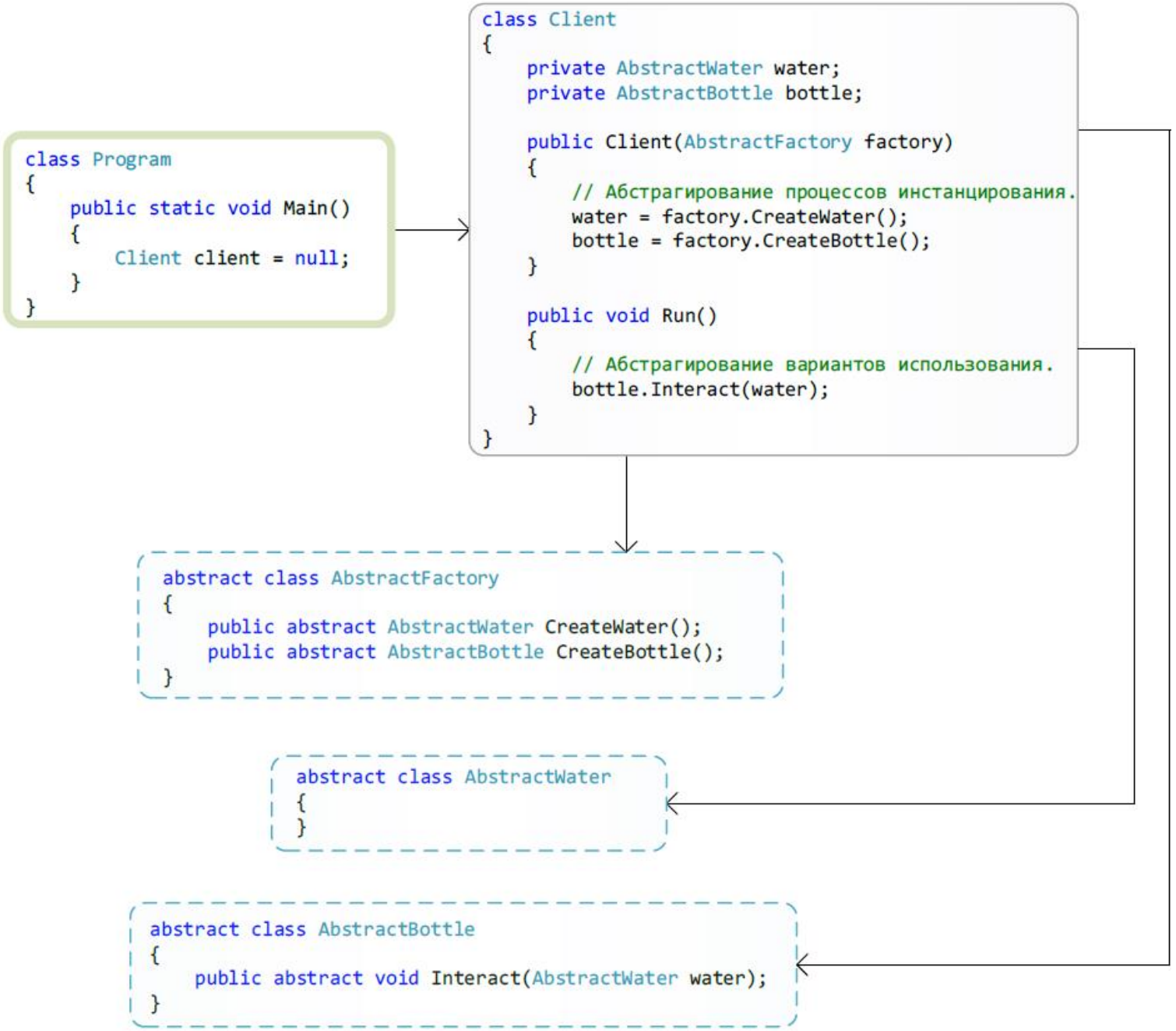
Абстрактные продукты, фабрики, клиент



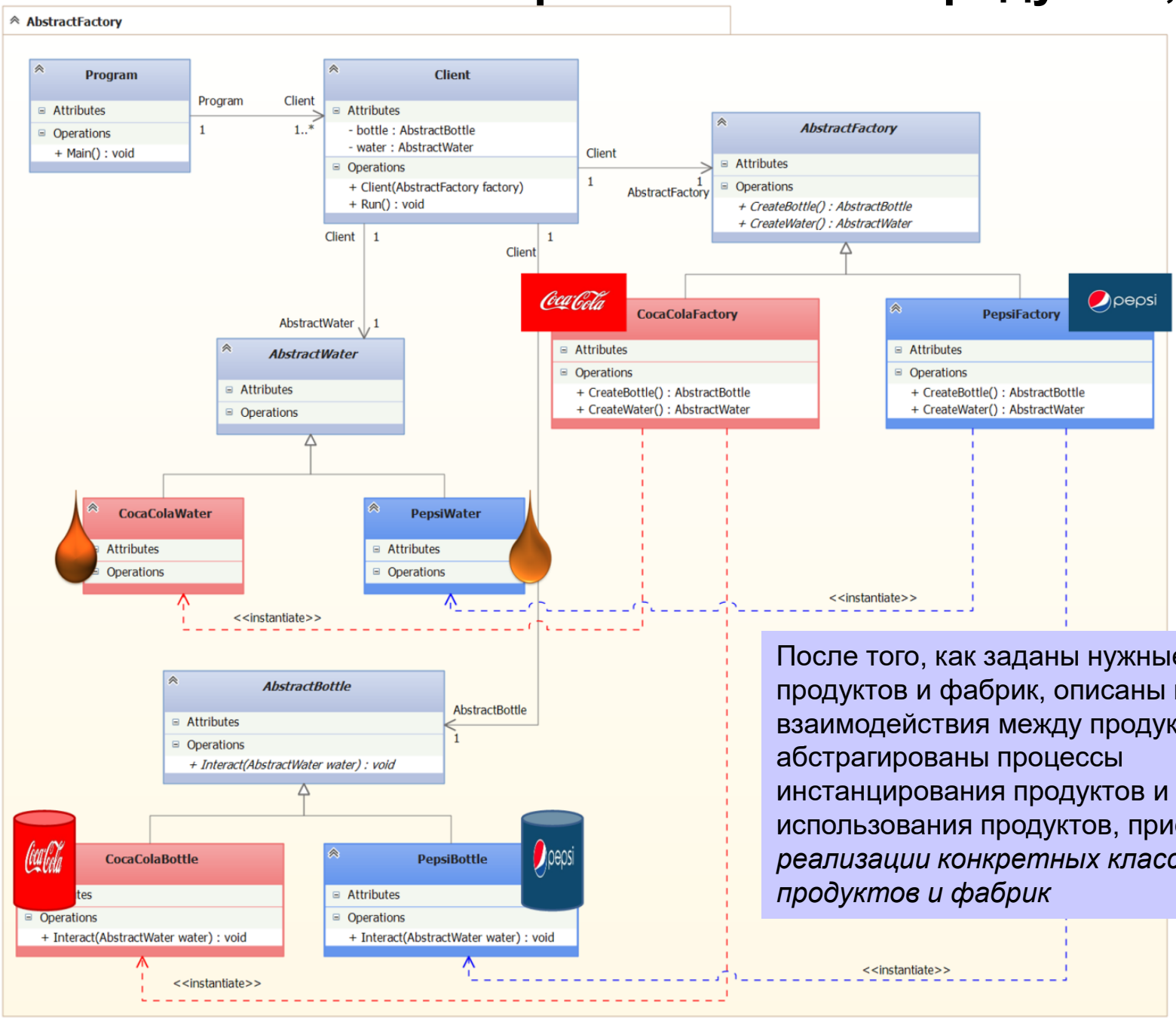
Вначале необходимо создать абстрактные классы для задания типов продуктов (AbstractWater и AbstractBottle) и типа фабрик (AbstractFactory). Описать интерфейсы взаимодействия с каждым типом продукта и фабрики.

Далее требуется создать конкретный класс Client в котором абстрактно (без реализации) описать процессы порождения экземпляров типов продуктов и варианты использования этих типов продуктов, через имеющиеся у них абстрактные интерфейсы. Также класс Client реализует идею инкапсуляции вариаций (сокрытие частей программной системы).

Абстрактные продукты, фабрики, клиент

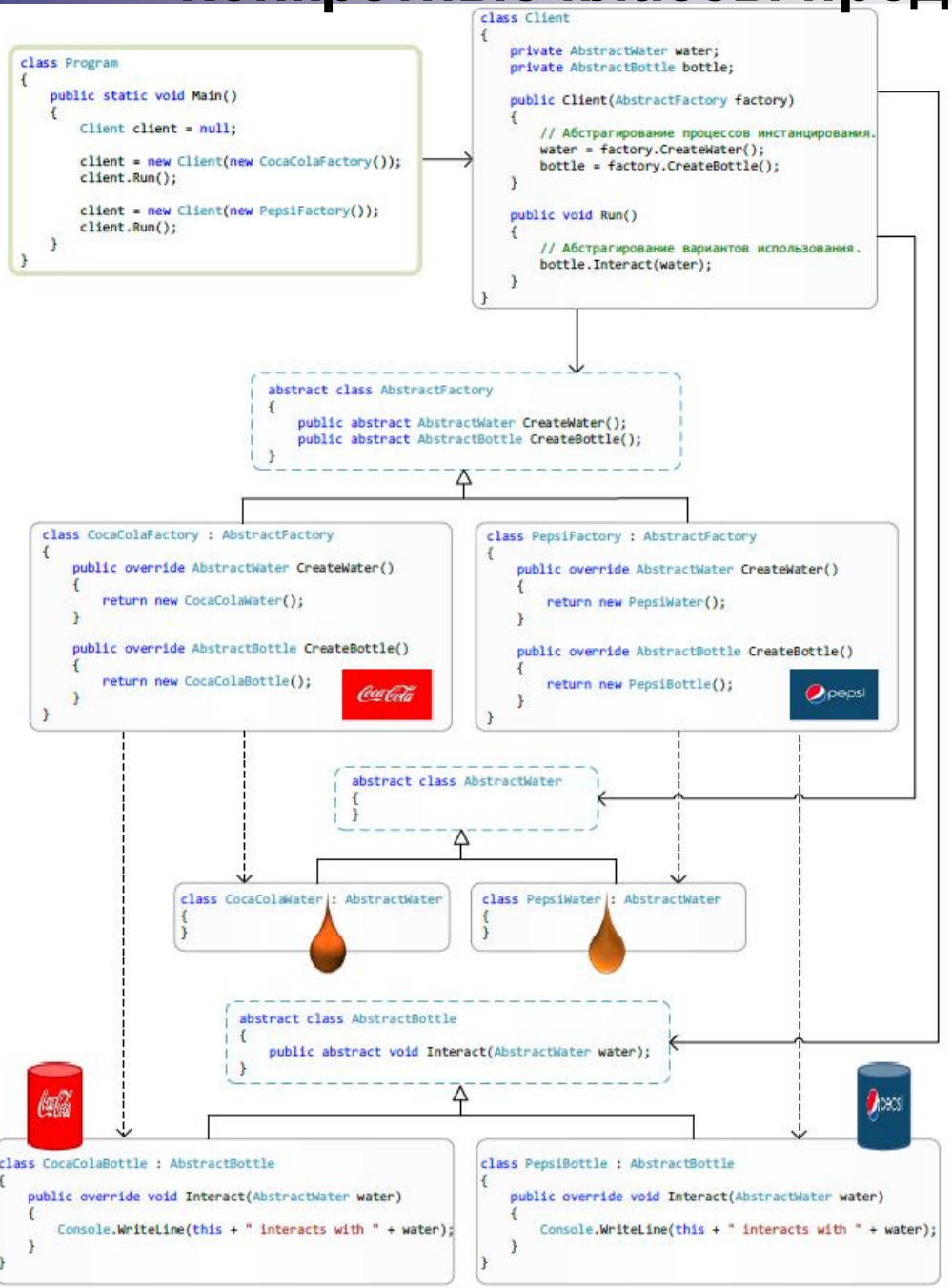


+ Конкретные классы продуктов, фабрик

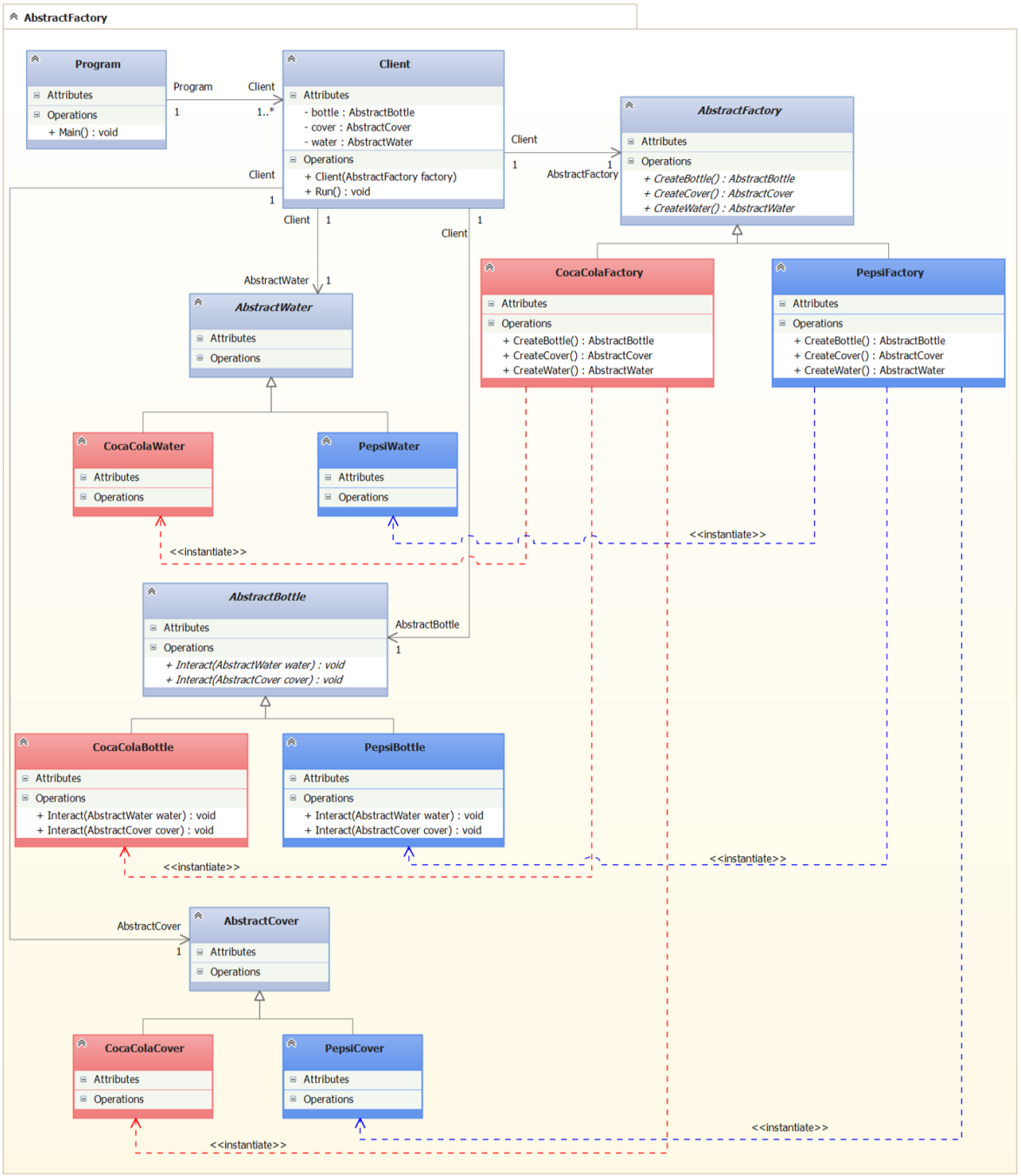


После того, как заданы нужные типы продуктов и фабрик, описаны интерфейсы взаимодействия между продуктами, абстрагированы процессы инстанцирования продуктов и варианты использования продуктов, приступают к *реализации конкретных классов продуктов и фабрик*

+ Конкретные классы продуктов, фабрик



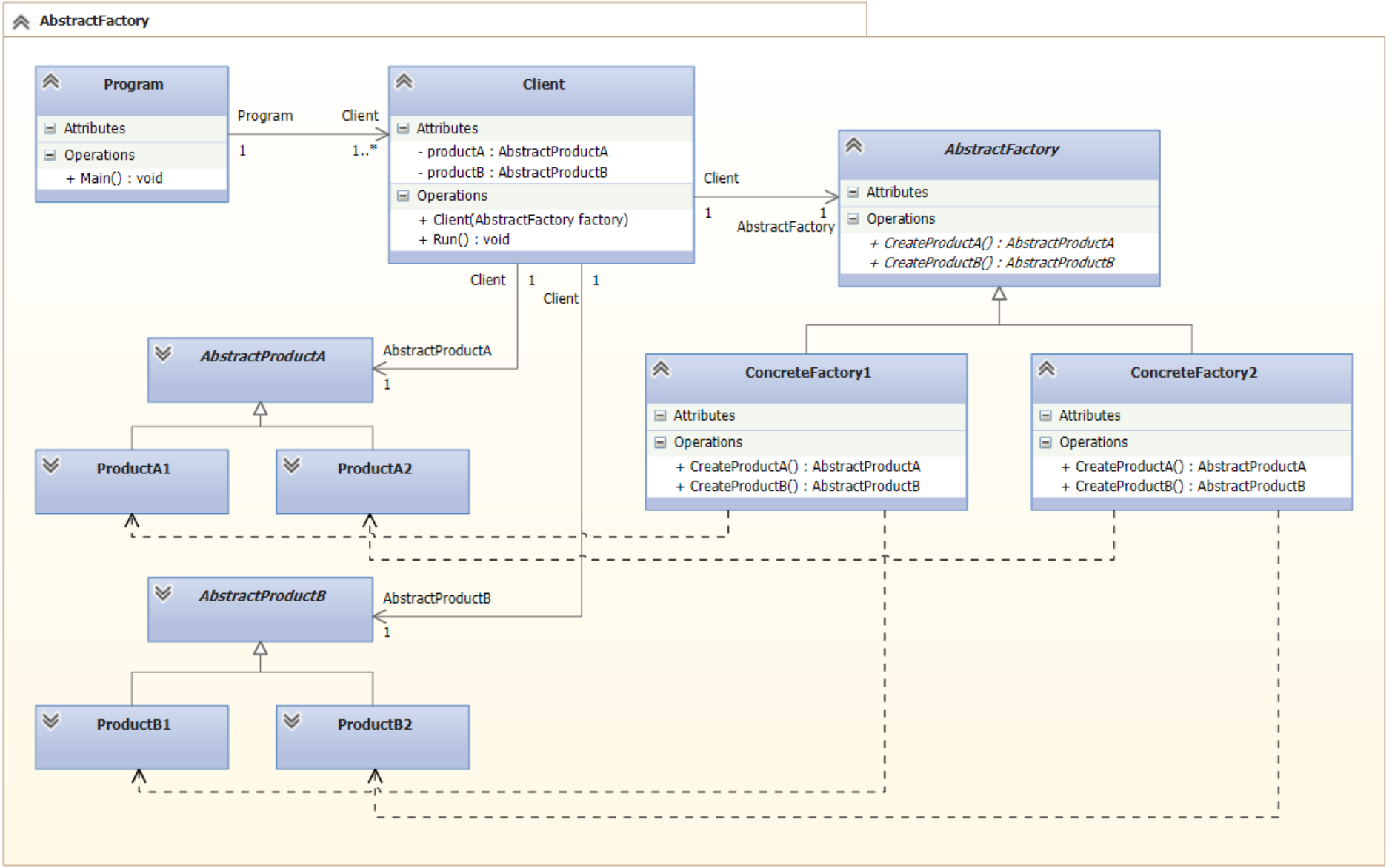
Добавление новых видов продуктов (крышки, этикетки, тара)



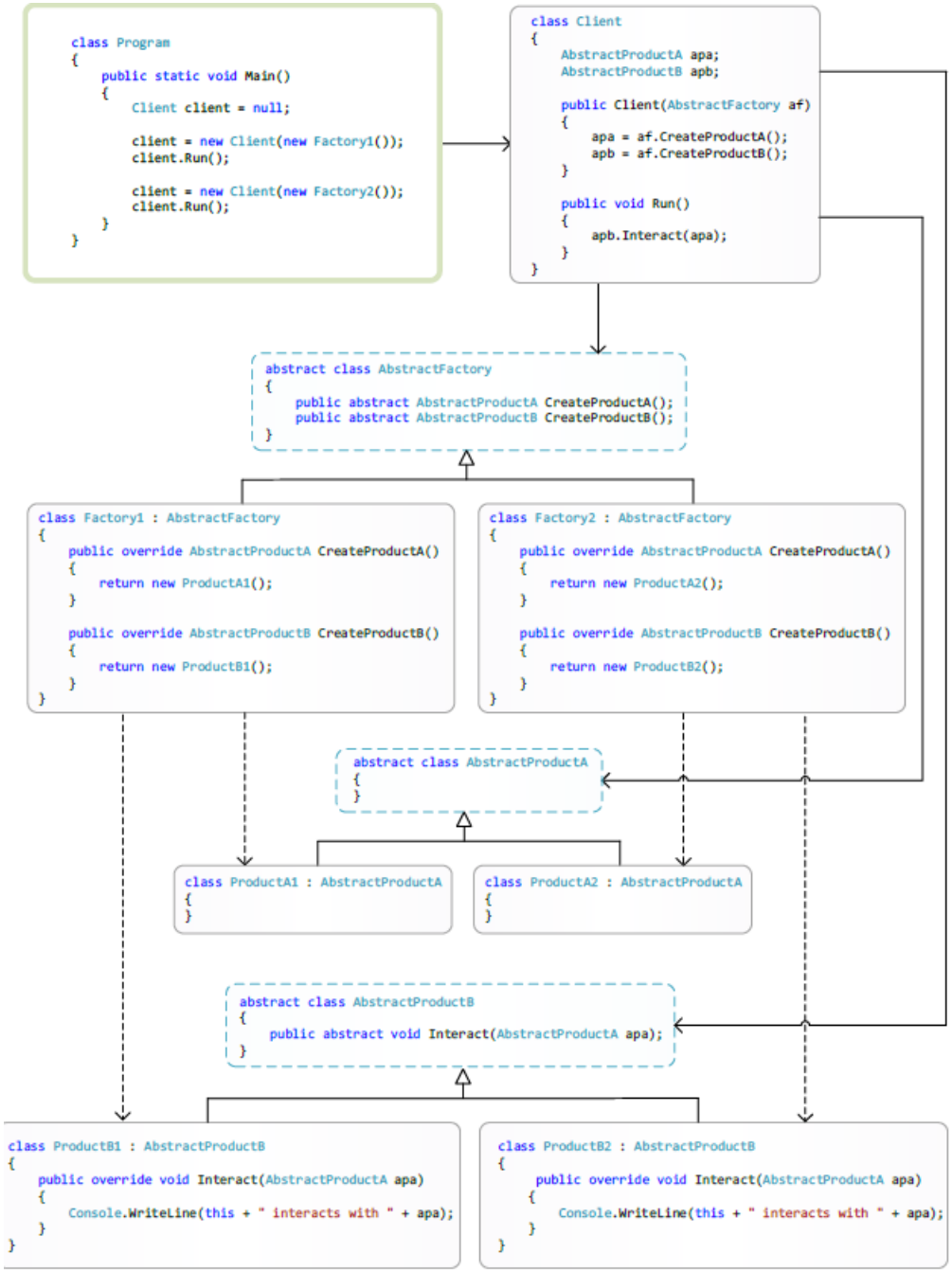
Шаги реализации

1. Создать таблицу соотношений типов продуктов к вариантам семейств продуктов.
2. Свести все вариации продуктов к общим интерфейсам.
3. Определить интерфейс абстрактной фабрики. Он должен иметь фабричные методы для создания каждого из типов продуктов.
4. Создать классы конкретных фабрик, реализовав интерфейс абстрактной фабрики. Этих классов должно быть столько же, сколько и вариаций семейств продуктов.
5. Изменить код инициализации программы так, чтобы она создавала определённую фабрику и передавала её в клиентский код.
6. Заменить в клиентском коде участки создания продуктов через конструктор вызовами соответствующих методов фабрики.

Структура паттерна на языке UML



Структура паттерна на языке C#



Участники

AbstractProduct - **Абстрактный продукт**:
Абстрактные классы продуктов предоставляют абстрактные интерфейсы взаимодействия с объектами-продуктами производных конкретных классов.

AbstractFactory - **Абстрактная фабрика**:
Класс AbstractFactory содержит в себе набор абстрактных фабричных методов. Эти абстрактные методы описывают интерфейс взаимодействия с объектами-фабриками и имеют возвращаемые значения типа абстрактных-продуктов, тем самым предоставляя возможность применять технику абстрагирования процесса инстанцирования. Класс AbstractFactory не занимается созданием объектов-продуктов, ответственность за их создание ложится на производный класс ConcreteFactory.

Client - **Клиент**:
Класс Client создает и использует продукты, пользуясь исключительно интерфейсом абстрактных классов AbstractFactory и AbstractProduct и ему ничего не известно о конкретных классах фабрик и продуктов.

ConcreteProduct - **Конкретный продукт**:
Конкретные классы продукты, наследуются от абстрактных классов продуктов. Объекты-продукты конкретных классов предполагается создавать в телах фабричных методов реализаций соответствующих фабрик.

ConcreteFactory - **Конкретная фабрика**:
Классы конкретных фабрик, наследуются от абстрактной фабрики и реализуют фабричные методы порождающие объекты-продукты.

Отношения между участниками

Отношения между классами

- Класс Client имеет отношения ассоциации с классами абстрактных продуктов и классом абстрактной фабрики.
- Все конкретные классы продуктов имеют отношения наследования с абстрактными классами продуктов.
- Все конкретные классы фабрик имеют отношения наследования с классом абстрактной фабрики и связи отношения зависимости (стереотипа - instantiate) с конкретными классами порождаемых продуктов.

Отношения между объектами

- В системе создается (чаще всего) только один экземпляр конкретной фабрики. Задачей конкретной фабрики является создание объектов продуктов, входящих в определенное семейство.
- При создании экземпляра клиента, клиент конфигурируется экземпляром конкретной фабрики (ссылка на экземпляр фабрики передается в качестве аргумента конструктора клиента).
- Абстрактный класс AbstractFactory передает ответственность за создание объектов-продуктов производным конкретным фабрикам.

Применимость паттерна

Паттерн **Abstract Factory** рекомендуется использовать, когда:

- Требуется создавать объекты-продукты разных типов и налаживать между ними взаимодействие, при этом образуя семейства из этих объектов-продуктов. Входящие в семейство объекты-продукты обязательно должны использоваться вместе.
- Требуется построить подсистему (модуль или компонент) таким образом, чтобы ее внутреннее устройство (состояние и/или поведение) настраивалось при ее создании. При этом чтобы ни процесс, ни результат построения подсистемы не был зависим от способа создания в ней объектов, их композиции (составления и соединения объектов) и представления (внутреннего состояния объектов).
- Подсистема или система должна настраиваться (конфигурироваться) через использование одного из семейств объектов-продуктов, порождаемых одним объектом-фабрикой;

Преимущества и недостатки

+ Гарантирует сочетаемость создаваемых продуктов.

+ Избавляет клиентский код от привязки к конкретным классам продуктов.

+ Выделяет код производства продуктов в одно место, упрощая поддержку кода.

+ Упрощает добавление новых продуктов в программу.

+ Реализует *принцип открытости/закрытости*.

- Усложняет код программы из-за введения множества дополнительных классов.

- Требует наличия всех типов продуктов в каждой вариации.